

Scorm: Control Abstraction

Lí thuyết

- **Deep binding** → Biến luôn dùng môi trường mà nó định nghĩa ban đầu. Môi trường được xác định lúc truyền tham số hàm. (tức sử dụng môi trường của **scope cha**)
- **Shallow binding** → Môi trường của hàm được quyết định **tại thời điểm hàm được thực thi**, (tức sử dụng môi trường của hàm chứa nó)

Cơ chế gọi chương trình con

- **Simple Call Return (Gọi trở về đơn giản)**
 - Không cho phép gọi đệ quy (Thường thấy trong assembly/Script Window)
 - Lệnh gọi phải tường minh
 - Chỉ có một điểm để gọi chương trình, không thể chen ngang
 - Truyền tức thời/thực thi tức thời. Chương trình gọi sẽ ngưng để chương trình được gọi thực thi. Một thời điểm chỉ có 1 chương trình chạy.
- **Recursive Call (gọi đệ quy)**
 - Có 2 dạng: đệ quy trực tiếp (gọi lại chính nó) và đệ quy gián tiếp (gọi lại chính nó thông qua chương trình khác $A \rightarrow B \rightarrow C \rightarrow A$)
 - Có lệnh gọi tường minh
 - Chỉ có duy nhất 1 điểm đầu vào, không thể chen ngang
 - Một thời điểm chỉ có 1 chương trình chạy
- **Exception Processing Handler (biến cố và xử lý biến cố)**
 - Không có lệnh gọi tường minh
 - Hướng đến lập trình sự kiện (click chuột, nhấn chữ A sẽ làm gì ...)
 - Xử lý biến cố (Error Handler): ném ra lỗi khi chia cho 0 ...
- **Coroutines (Song hành/Cộng hành)**
 - Ở một thời điểm chỉ có một chương trình chạy.
 - Có thể có nhiều điểm vào, gây cảm giác chạy đồng thời
- **Tasks (Công tác)**
 - Thực thi đồng thời với nhiều task khác nhau, tức nhiều chương trình con chạy đồng thời.
 - Thực hiện trên máy có đa vi xử lý hoặc đơn vi xử lý có time sharing
- **Scheduled SubProgram (Định thời)**

- Khi A gọi B thì điều khiển không nhất thiết phải chuyển lập tức từ A qua B, mà có thể trì hoãn thời gian tùy theo chúng ta định thời (10s, 20s...) → Định thời theo thời gian
- Ngoài ra còn định thời theo độ ưu tiên, độ ưu tiên cao hơn sẽ thực thi trước.
- Bộ định thời quyết định một chương trình chạy lúc nào.
- Application: Lập trình thời gian thực (real-time), Lập trình định thời trong hệ điều hành.

Cơ chế truyền thông số

- Input-Output → vừa để truyền vào vừa để truyền ra.
 - By value-result
 - By reference
 - By name
- Input Only → chỉ dùng để truyền vào cho chương trình con
 - By value
 - By constant reference
- Output Only → chỉ nhận dữ liệu từ bên trong chương trình con trả ra.
 - By result
 - As a result of a function

Quiz

Câu 1:

Cho đoạn chương trình sau viết bằng Pascal:

```
var X : real;

procedure SUB2(X, Y : real; function F(U : real) : real);
var Z : real;
begin
  Z := abs(X - Y);
  Z := (F(X) + F(Y)) * Z / 2;
  write(Z);

procedure SUB1;
var Y : real;

  function FUNC(V : real) : real;
  begin
    FUNC := X * V + Y;

begin
  Y := 1;
  SUB2(1, 2, FUNC);

begin
  X := 3;
```

SUB1;
end.

- a. Pascal là ngôn ngữ static-scoping, vậy giá trị z được in ra trong chương trình con SUB2 là?
_____ **5.5** _____
- b. Giả sử chương trình trên được viết trên một ngôn ngữ dynamic scoping dùng shallow binding thì giá trị z được in ra khi SUB2 được thực thi là? _____ **3.5** _____
- c. Giả sử chương trình trên được viết trên một ngôn ngữ dynamic scoping dùng deep binding thì giá trị z được in ra khi SUB2 được thực thi là? _____ **5.5** _____

Giải:

- a. $X=3, Y=1$ đưa vào SUB2 $\rightarrow \text{FUNC}(V) = 3V + 1$. (FUNC không nhìn thấy 1,2)
Lúc này đi vào tính SUB2 $\rightarrow$ Gán lại $X=1, Y=2$. Tính: $Z = \text{abs}(X-Y) = \text{abs}(1-2) = 1$.
Tiếp tục tính: $Z = (F(X) + F(Y)) * Z / 2$; mà $F(X) = F(1) = \text{FUNC}(1) = 3*1 + 1 = 4$, tương tự $F(Y) = 3*2 + 1 = 7$.
 $\rightarrow Z = (4 + 7) * 1 / 2 = 5.5$.
- b. Dùng shallow binding $\rightarrow$ FUNC dùng môi trường của hàm chứa nó (SUB2) $\rightarrow \text{FUNC} = 1*V + 2$.
Tính toán SUB 2 $\rightarrow Z = (3 + 4) * 1 / 2 = 3.5$
- c. Dùng Deep binding $\rightarrow$ FUNC dùng môi trường của SUB1, tương tự câu A $\rightarrow$ FUNC là $3V + 1$, tính toán SUB2 ra kết quả 5.5

Câu 2.

Cho đoạn chương trình sau viết bằng Pascal:

var X : real;

procedure SUB2(X, Y : real; function F(U : real) : real);

var Z : real;

begin

Z := abs(X - Y);

Z := (F(X) + F(Y)) * Z / 2;

write(Z);

procedure SUB1;

var Y : real;

function FUNC(V : real) : real;

begin

FUNC := X * V + Y;

begin

Y := 2;

SUB2(1, 2, FUNC);

begin

X := 4;

SUB1;
end.

- a. Pascal là ngôn ngữ static-scoping, vậy giá trị z được in ra trong chương trình con SUB2 là?
_____ **8.0** _____
- b. Giả sử chương trình trên được viết trên một ngôn ngữ dynamic scoping dùng shallow binding thì giá trị z được in ra khi SUB2 được thực thi là? _____ **3.5** _____
- c. Giả sử chương trình trên được viết trên một ngôn ngữ dynamic scoping dùng deep binding thì giá trị z được in ra khi SUB2 được thực thi là? _____ **8.0** _____

Giải: Tương tự câu trên, đổi số.

- a. Static scoping → FUNC có môi trường của cha nó (SUB1) → $X=4, Y=2. \rightarrow FUNC = 4V + 2 \rightarrow Z = (FUNC(1) + FUNC(2)) * 1/2 = (6 + 10) * 1/2 = 8.0$
- b. shallow binding → FUNC có môi trường của SUB2 → $FUNC = 1 * V + 2 \rightarrow$ đáp án như câu 1b → 3.5
- c. Static scoping và deep binding cho kết quả giống nhau. Kết quả giống câu a → 8.0

Câu 3.

Trong các cơ chế gọi chương trình con,

1. Cơ chế _____ **song hành (coroutines)** _____ cho phép có thể chuyển điều khiển từ chương trình gọi sang một điểm không phải là điểm bắt đầu của chương trình được gọi
2. Cơ chế _____ **Biến cố (Exception)** _____ cho phép có thể chuyển điều khiển từ chương trình gọi sang chương trình được gọi mà không có lệnh gọi tương minh?
3. Cơ chế _____ **Công tác (tasks)** _____ cho phép chương trình được gọi thực hiện tiếp lệnh kế tiếp sau lệnh gọi mà không chờ chương trình được gọi kết thúc?
4. Cơ chế gọi chương trình con _____ **Đệ quy (Recursion)** _____ khi hiện thực sẽ tạo ra nhiều bản hoạt động của cùng một chương trình con khi thực thi?

Câu 4.

Trong các cơ chế truyền thông số,

1. Cơ chế _____ **As a result of a function** _____ không có thông số thực
2. Cơ chế _____ **Pass by name** _____ không tính toán thông số thực mà truyền mã của thông số thực cho thông số hình thức
3. Cơ chế _____ **Pass by constant reference** _____ buộc chương trình được gọi không thể thay đổi giá trị thông số hình thức
4. Cơ chế _____ **Pass by value** _____ ngăn chặn những thay đổi trên thông số hình thức dẫn đến thay đổi trên thông số thực sau khi kết thúc chương trình được gọi

Câu 5.

- a. Khi lập trình cho các giao diện Windows, người lập trình sẽ định nghĩa các phương thức sẽ được thực thi khi một nút nhấn được chọn, một văn bản được nhập,... Cơ chế **Exception** là cơ chế gọi chương trình con chủ yếu được sử dụng trong trường hợp này.
- b. Khi lập trình cho một hệ thống đòi hỏi các đáp ứng phải xảy ra đúng theo các mốc thời gian bất kể máy chạy nhanh hoặc chậm, cơ chế **Scheduled Subprogram** là cơ chế gọi chương trình con cần được sử dụng trong trường hợp này.
- c. Hiện tượng tương tranh (race condition) hoặc khoá chết (deadlock) có thể xảy ra khi dùng cơ chế gọi chương trình con **Tasks**
- d. Cơ chế **Exception** thường được sử dụng để việc xử lý lỗi được tách bạch khỏi logic của chương trình

Câu 6.

Trong các cơ chế gọi chương trình con, cơ chế nào cho phép có thể chuyển điều khiển từ chương trình gọi sang một điểm không phải là điểm bắt đầu của chương trình được gọi?

- a. **Song hành (Coroutine)**
- b. Công tác (Task)
- c. Biến cố (Exception)
- d. Định thời (Scheduled)

Câu 7.

Trong các cơ chế gọi chương trình con, cơ chế nào cho phép chương trình gọi thực hiện tiếp lệnh kế tiếp sau lệnh gọi mà không chờ chương trình được gọi kết thúc?

- a. Định thời (Scheduled)
- b. Biến cố (Exception)
- c. **Công tác (Task)**
- d. Song hành (Coroutine)

Câu 8.

Bản hoạt động của một chương trình con trong cơ chế Gọi đệ qui (Recursive call) ít nhất cần phải chứa các liên kết gì?

- a. **Liên kết đến lệnh sau lệnh gọi trong mã của chương trình con gọi (caller) chương trình con hiện tại (callee)**

b. Liên kết đến bản hoạt động của chương trình con bao lấy chương trình con hiện tại trong văn bản chương trình

c. Liên kết đến bản hoạt động của chương trình con (caller) gọi chương trình con hiện tại (callee)

d. Liên kết đến bản hoạt động của chương trình con được gọi bởi chương trình con hiện tại

Câu 9.

Trong các cơ chế truyền thông số, cơ chế nào làm thay đổi thông số thực ngay khi thông số hình thức bị thay đổi mà không chờ đến khi chương trình được gọi kết thúc?

a. Pass by value-result

b. Pass by result

c. Pass by reference

d. Pass by name

Câu 10.

Khi lập trình cho một hệ thống đòi hỏi các đáp ứng phải xảy ra đúng theo các mốc thời gian bất kể máy chạy nhanh hoặc chậm, cơ chế gọi chương trình con nào cần được sử dụng?

a. Định thời (Scheduled)

b. Song hành (Coroutine)

c. đệ qui (Recursive Call)

d. Biến cố (Exception)

Trong các cơ chế gọi chương trình con, cơ chế nào cho phép có thể chuyển điều khiển từ chương trình gọi sang chương trình được gọi mà không có lệnh gọi tường minh?

Select one:

☐ a. Định thời (Scheduled)

☐ b. Song hành (Coroutine)

☐ c. Công tác (Task)

☒ d. Biến cố (Exception)

[Clear my choice](#)